Empirical Asset Pricing via Machine Learning

Forecasting with Machine Learning

Natasia Villadsen

2026-11-09

Introduction

Today

1	**The paper**	Methodology of the paper, and the associated machine learning notions
2	**The replication**	What would it take to reproduce the main table: data, compute, and code?
3	**The gaps**	What can we reproduce, what do we not manage to reproduce, and why?

> Research rarely fits into the neat box of implementation as laid out in Lectures 3 and 4

The Paper

Empirical Asset Pricing via Machine Learning

- Title: "Empirical Asset Pricing via Machine Learning", Gu et al., 2020, *Review of Financial Studies*

Authors:

Shihao Gu University of Chicago, Booth School of Business

Bryan Kelly AQR Capital Management, NBER, and Yale University

Dacheng Xiu University of Chicago, Booth School of Business

Research question: Can machine learning methods improve forecasting of asset returns?

Empirical Asset Pricing

- Asset pricing asks why some assets earn higher expected returns than others, and why expected returns change over time
- The risk premium is the expected return in excess of the risk-free rate, the compensation for bearing risk
- It is unobserved. We observe the realized returns, which are dominated by unforecastable news
- Measuring the risk premium is therefore a forecasting problem, e.g., forecast the excess returns of next month using information available today

> Many correlated predictors and an unknown functional form make this a natural application for machine learning

Forecasting Target (Sec. 1)

- The authors aim to forecast the excess returns of an asset
- Construct an “additive prediction-error model”,

$$r_{i,t+1} = \underbrace{\mathbb{E}_t\left[r_{i,t+1}\right]}_{g^*(z_{i,t})} + \varepsilon_{i,t+1}$$

- $r_{i,t+1}$ is the excess return (over the T-bill rate) of stock i at time $t+1$
- $g^*(\cdot)$ is the forecasting function, the conditional expected return (the risk premium)
- $z_{i,t}$ is the information set (predictors) for stock i at time t
- $\varepsilon_{i,t+1}$ is the error term

Predictors (Sec. 2.1)

STOCK CHARACTERISTICS

94 characteristics $c_{i,t}$ from the cross-section literature

MACRO VARIABLES

8 variables x_t from Goyal and Welch (2008): dp, ep, bm, ntis, tbl, tms, dfy, svar

INDUSTRY

74 dummy variables, one per two-digit SIC industry code

Stock characteristics are made to interact with the macro variables,

$$z_{i,t} = (1, x_t^\intercal)^\intercal \otimes c_{i,t}, \qquad p = 94 \times (8+1) + 74 = 920$$

Here p is the number of predictors

dp: dividend-price ratio, ep: earnings-price ratio, bm: book-to-market ratio, ntis: net equity expansion, tbl: Treasury-bill rate, tms: term spread, dfy: default spread, svar: stock variance

The Paper in Numbers

920

baseline predictors per stock and month

~30,000 stocks

monthly returns, 1957–2016

−3.46% vs. 0.40%

monthly R^2_{oos}: OLS with all predictors vs. the best trees and neural networks

> Forecasting excess returns is a low signal-to-noise problem: a monthly R^2_{oos} of a fraction of a percent can be economically meaningful (Campbell and Thompson, 2008)

The Authors' Main Findings

Linear models break down with many predictors

OLS with all 920 predictors: $R^2_{\text{oos}} = -3.46\%$ per month, worse than forecasting zero excess returns

Regularization repairs the linear model

Elastic net, PCR and PLS turn the R^2_{oos} positive (0.11% to 0.27%)

Non-linearity helps, via interactions

Trees and neural networks reach 0.33% to 0.40% (splines without interactions (GLM) do not surpass the best linear models)

Moderate depth networks win

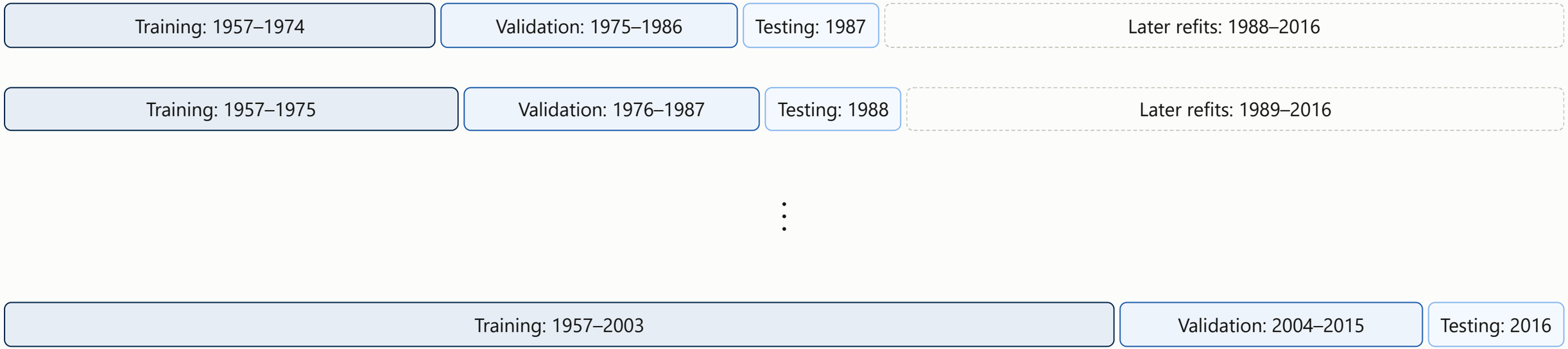
Neural network performance peaks at three hidden layers, outperforming all machine learning techniques

Tip

PCR and PLS compress the predictors into a few components. Both are introduced next lecture

Methodology of Paper

Sample Splitting (Sec. 2.1)



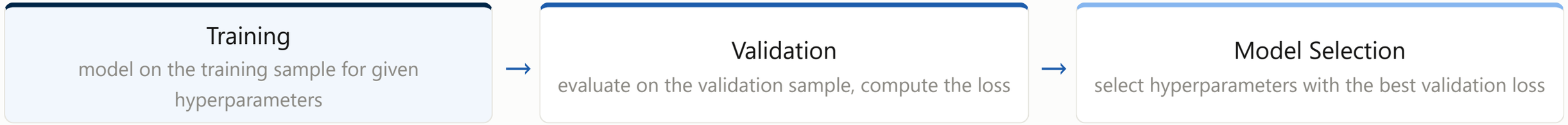
- The training sample is an expanding window, one more year at every refit
- The validation sample is a rolling window of fixed length, 12 years
- The models are refit once a year, due to computational overhead and low frequency of signal (data/predictor) arrival: 30 estimations instead of 360 over 30 test years
- Each fitted model is tested on the next year only

Hyperparameters and Validation (Sec. 1.1)

The regularization approaches of the paper rely on hyperparameters, e.g.,

- The penalty parameter (elastic net)
- The number of components (PLS, PCR)
- The tree depth (random forests, boosted trees)

They control model complexity, and there is little theoretical guidance on how to set them



Bias, Variance, and Regularization (Sec. 1.3)

Flexible models

- A flexible model (nonlinearities, interactions, deep trees, neural networks) can approximate a wide range of functions, so bias is low
- But if it is **too** flexible for the data, it has many parameters to estimate from limited information, so the estimates have high variance and are unstable
- It then fits noise (adapts to the current sample) rather than signal and performs poorly out of sample

Regularized models

- Accept some bias in return for lower variance
- By means of penalties (LASSO, ridge, elastic net), fewer dimensions (PCR and PLS), shallow trees, early stopping, or averaging over trees or seeds
- The degree of regularization is a hyperparameter and can be determined via validation

Evaluation: Out-of-Sample R^2 (Sec. 1.8)

Forecasts are evaluated on the test sample (out-of-sample), pooling all stocks and months,

$$R^2_{\text{oos}} = 1 - \frac{\sum (r_{i,t+1} - \hat{r}_{i,t+1})^2}{\sum r^2_{i,t+1}}$$

- Compares the squared forecast errors of the model with those of a forecast of zero
- A positive R^2_{oos} means that the model beats a zero-excess-returns forecast (difficult!)

> The benchmark must be motivated and specified by the researcher

The Models

OLS+H	Linear regression on all 920 predictors
OLS-3+H	Linear regression on size, book-to-market and momentum only
Elastic net+H	Penalized linear regression with both ℓ_1 and ℓ_2 penalties
PCR	Principal Components Regression on K principal components, the main directions of variation in the predictors
PLS	Partial Least Squares Regression on K components built to covary with the returns (target), not just with the predictors
GLM+H	Generalized Linear Model: Linear regression on piecewise-polynomial (spline) transformations of each predictor, no interactions
Trees	Tree-based models: Random forests average many deep trees, boosted trees (+H) sequentially add many shallow trees
Neural networks	Feed-forward neural networks, NN1 to NN5 models have one to five hidden layers

> +H marks models estimated with the Huber loss: squared error for small forecast errors and absolute error for large ones

Forecast Accuracy in the Paper (Sec. 2.2)

Monthly out-of-sample R^2 (percent) – **Gu et al., 2020 Table 1**

Model	All	Top 1,000	Bottom 1,000
OLS+H, all 920 predictors	−3.46	−11.28	−1.30
OLS-3+H	0.16	0.31	0.17
Elastic net+H	0.11	0.25	0.20
PCR	0.26	0.06	0.34
PLS	0.27	−0.14	0.42
GLM+H	0.19	0.14	0.30
Random forest	0.33	0.63	0.35
Boosted trees+H	0.34	0.52	0.32
NN3 (best)	0.40	0.70	0.45

+H: Huber loss. Top and Bottom 1,000: the largest and smallest stocks by market value each month.

NN1 to NN5 range from 0.33 to 0.40 (All), 0.49 to 0.70 (Top) and 0.38 to 0.47 (Bottom)

Replication

Replicating Gu et al. (2020)

Replication is ultimately unsuccessful, as the paper omits key details. We are not aware of any successful, publicly available replication.

Step	Task	Hurdle
1 Data	Stock returns from CRSP and accounting data from Compustat, both through WRDS	Proprietary data + 1:1 matching
2 Pre-processing	Build 94 characteristics, rank them to $[-1, 1]$, lag them, fill in missing values, interact them with the macro variables and add industry dummies	Many choices are not fully stated, post-hoc updates
3 Estimation	Thirteen models with tuning grids, refit once a year over 30 test years	Computing time and memory
4 Evaluation	R^2_{OOS}	Mismatch of results

> The raw return data is proprietary (WRDS licence), so I cannot distribute the data

The Replication Package

> Journals increasingly require authors to publish their **replication code** in full with the paper

The authors have published code for the Monte Carlo simulations in Internet Appendix A, but not for the empirical study.

- Everything that the paper and the Internet Appendix do not state has to be decided by us
- Each such choice can affect the results (next slides)

The Scale of the Problem

3.76M
stock-months in our file, 1957–2016

94 → 920
characteristics → predictors, after the macro interactions and industry dummies

≈ 3 GB → 14 GB
the characteristics file, and the predictor matrix in single precision (3,762,139 × 921 × 4 bytes)

30
refits per model and hyperparameter value, one per test year

> Estimating all thirteen models needs a high-performance computing (HPC) cluster (a supercomputer)

Approaching the Data Set

We verify that our data match the description in the paper, asking: Do the discrepancies stem from the data?

Coverage

- 29,925 distinct stocks in our file and “almost 30,000” in the paper
- Stocks per month are 5,225 in our file and “exceeding 6,200” in the paper
- CRSP’s monthly file itself gives about 5,590

Returns

- 3,742,884 stock-months appear in both our file and CRSP
- 99.93% of these match CRSP’s return to within 0.0001 (correlation 0.9992)

Lacking Information

Solver for OLS + Huber

OLS with Huber loss has no analytical solution and must be minimized numerically. The paper does not state the algorithm applied

Scaling of the predictors

Not specified. Z-score standardization is the natural default, yet this cannot be assumed

Example: We observe that PLS for Top 1,000 is −0.62% under Z-score std. and −0.30% unscaled, against −0.14% in Gu et al., 2020

Grid of hyperparameters

Table A.5 lists K as the hyperparameter for PCR and PLS but gives no grid. We searched $K = 1 \text{ to } 100$

Our Replication vs. Table 1

Monthly R^2_{oos} (percent): All / Top 1,000 / Bottom 1,000. Full panel, test years 1987–2016

Model	This replication	Paper, Table 1
OLS-3	0.20 / 0.23 / 0.19	0.16 / 0.31 / 0.17
OLS	−141.38 / −491.65 / −52.20	−3.46 / −11.28 / −1.30
OLS, stopped early (calibrated)*	−3.46 / −13.28 / −0.95	−3.46 / −11.28 / −1.30
Elastic net+H	0.17 / 0.16 / 0.25	0.11 / 0.25 / 0.20
PCR	0.29 / 0.06 / 0.37	0.26 / 0.06 / 0.34
PLS	0.18 / −0.62 / 0.40	0.27 / −0.14 / 0.42
GLM, trees, neural networks	…	…

* Gradient descent stopped at a tolerance chosen to match −3.46 for *All*. Calibration, not replication. Top and Bottom still differ. PCR and PLS use z-score standardized predictors

> GLM, trees and neural networks were not run: given the issues so far and the HPC requirement, there is little reason to do so

What We Can and Cannot Reproduce

Same order of magnitude

- ✓ OLS-3: 0.20% vs. 0.16%
- ✓ PCR: 0.29% vs. 0.26%
- ✓ Elastic net: 0.17% vs. 0.11%
- ✓ The pattern: OLS-3 and the regularized linear models are positive, the full OLS is not

Does not reproduce

- ✗ OLS: −141.38% vs. −3.46%
- ✗ PLS: 0.18% vs. 0.27% (All) and −0.62% vs. −0.14% (Top 1,000)
- ✗ Stocks per month: 5,225 vs. more than 6,200

Overall, the replication can be considered unsuccessful, that is:
The results of the paper are **not reproducible with the information given**

Up to you

> You are the referee. The paper reports −3.46% R^2_{OOS} for OLS and we get −141.38%.
> Which three questions might you ask the authors, and which one matters most?

Explaining Why

Why Does OLS Do Worse Than the Other Models?

Two sources of forecast error: **bias** (the model systematically misses part of the true relation) and **variance** (the estimates are unstable across samples),

OLS

920 coefficients

Low bias, but high variance: with noisy returns the coefficients are poorly determined, so the forecasts are unstable too. Paper: $R^2_{\text{oos}} = -3.46\%$

OLS-3

3 coefficients

Low variance, but the other 917 predictors are ignored, so the bias is higher. Paper: $R^2_{\text{oos}} = 0.16\%$

REGULARIZED MODELS

Shrinkage or compression

Use the predictors, but shrink the coefficients or compress them into a few components: a little bias for much lower variance. Paper: $R^2_{\text{oos}} = 0.11\%$ to 0.27%

Why Does Our OLS Differ From the Paper's?

Our OLS gives -141.38%, the paper's -3.46%. Three differences between our implementation and theirs could explain this,

Difference	Possible mechanism	Status
Solver for the Huber loss (start, step size, stopping rule)	Stopping early shrinks the coefficients	Matches *All* once calibrated
Predictor construction (scaling, missing values)	Changes which columns are extreme or near-constant (potential source of instability)	Not tested
Data	Different sample of stocks: 5,225 vs. more than 6,200 per month	Not tested

> We matched the paper's number by calibrating the solver, but a match does not prove the cause

Wrapping Up

What the Replication Shows

THE PICTURE HOLDS

Full OLS fails, while OLS-3 and the regularized models do well

THE NUMBERS DO NOT

Our numerical results differ from those of the paper, the OLS results in particular

UNSTATED CHOICES MATTER

Choices left open, such as the solver and the scaling, change the results a lot

> In your own work, document every implementation choice and report how sensitive the results are to it

For Discussion

1	**Complexity and Noise**	The paper finds that a shallow network beats deeper ones. Why would more depth hurt when data is noisy?
2	**Evidence**	Which rows of Table 1 would you trust most? Compare All, Top 1,000 and Bottom 1,000

Resources

- Campbell, J. Y. and Thompson, S. B. (2008). Predicting excess stock returns out of sample: Can anything beat the historical average? *Review of Financial Studies*, 21(4), 1509–1531
- Gu, S., Kelly, B. and Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273
- Internet Appendix of the paper: simulation, algorithms, sample splitting, hyperparameters
- Authors' Monte Carlo code: github.com/xiubooth/ML_Codes
- James et al. (2023), An Introduction to Statistical Learning with Applications in Python: penalized regression, PCR and PLS, trees
- Lewellen, J. (2015). The cross-section of expected stock returns. *Critical Finance Review*. The OLS-3 benchmark

Appendix

Early Stopping/Termination Is Regularization

Our OLS by accelerated gradient descent from zero (monthly R^2_{oos}, percent)

Tolerance	Iterations (median)	All	Top 1,000	Bottom 1,000
1e-5	369	−12.86	−50.09	−5.29
3e-5	187	−8.06	−32.72	−3.12
1e-4	107	−4.95	−19.06	−1.56
2e-4	78	−3.46	−13.28	−0.95
5e-4	51	−1.92	−8.04	−0.32
1e-3	36	−1.03	−5.32	0.07
1e-2	11	0.07	−1.82	0.49

Fewer iterations shrink the coefficients toward zero, as early stopping does for networks (Sec. 1.7). This moves OLS to the paper's order of magnitude but nothing suggests this is the method of choice. We fail to select a tolerance to replicate both All, Top, and Bottom of the paper

Theory: The Variance of the OLS Estimator

$$y = X\beta + \varepsilon, \qquad \hat{\beta} = (X^\top X)^{-1} X^\top y, \qquad \mathrm{Var}(\hat{\beta} \mid X) = \sigma^2 (X^\top X)^{-1}$$

where the errors ε have variance σ^2

- If a column of X is almost all zeros, or almost a copy of other columns, the data say very little about its coefficient
- The variance of that coefficient estimate is then very large

> Little variation in a predictor and high correlation between predictors have the same effect: the coefficients are poorly determined

Theory: What Regularization Does

Ridge regression penalizes the size of the coefficients,

$$\hat{\beta}^{\text{ridge}} = (X^\intercal X + \lambda I)^{-1} X^\intercal y$$

- All coefficients are shrunk towards zero with a strength governed by λ ($\lambda = 0 \to$ OLS)
- Adding λI to $X^\intercal X$ makes the estimates less sensitive to the data, which lowers the variance at the cost of bias
- Early stopping has a similar effect: gradient descent starts with all coefficients at zero and moves towards the OLS solution, so stopping early leaves them closer to zero

Demo: Data

Python Code | Start Over | Run Code

```
import numpy as np
from sklearn.linear_model import RidgeCV
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
X = rng.standard_normal((1200, 110))  # 10 relevant and 100 useless predictors
y = X[:, :10] @ np.full(10, 0.3) + rng.standard_normal(1200)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.8,
                                                    shuffle=False)
```

Demo: OLS

Python Code | Start Over | Run Code

```
ols_10 = LinearRegression()
ols_110 = LinearRegression()

ols_10.fit(X_train[:, :10], y_train)
ols_110.fit(X_train, y_train)

r2_ols_10 = 1 - np.sum((y_test - ols_10.predict(X_test[:, :10])) ** 2) / np.sum(y_test ** 2)
r2_ols_110 = 1 - np.sum((y_test - ols_110.predict(X_test)) ** 2) / np.sum(y_test ** 2)
```

Demo: Ridge

Python Code | Start Over | Run Code

```
ridge_10 = RidgeCV(alphas=[0.1, 1, 10, 100, 1000])
ridge_110 = RidgeCV(alphas=[0.1, 1, 10, 100, 1000])

ridge_10.fit(X_train[:, :10], y_train)
ridge_110.fit(X_train, y_train)

r2_ridge_10 = 1 - np.sum((y_test - ridge_10.predict(X_test[:, :10])) ** 2) / np.sum(y_test ** 2)
r2_ridge_110 = 1 - np.sum((y_test - ridge_110.predict(X_test)) ** 2) / np.sum(y_test ** 2)
```

Demo: Results

Python Code | Start Over | Run Code

```
print("OLS R2_OOS, 10 predictors:    %.2f" % r2_ols_10)
print("OLS R2_OOS, 110 predictors:   %.2f" % r2_ols_110)
print("Ridge R2_OOS, 10 predictors:  %.2f" % r2_ridge_10)
print("Ridge R2_OOS, 110 predictors: %.2f" % r2_ridge_110)
```

```
OLS R2_OOS, 10 predictors:    0.44
OLS R2_OOS, 110 predictors:   0.01
Ridge R2_OOS, 10 predictors:  0.44
Ridge R2_OOS, 110 predictors: 0.25
```

> Ridge matches OLS with 10 predictors and rescues it with 110